

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

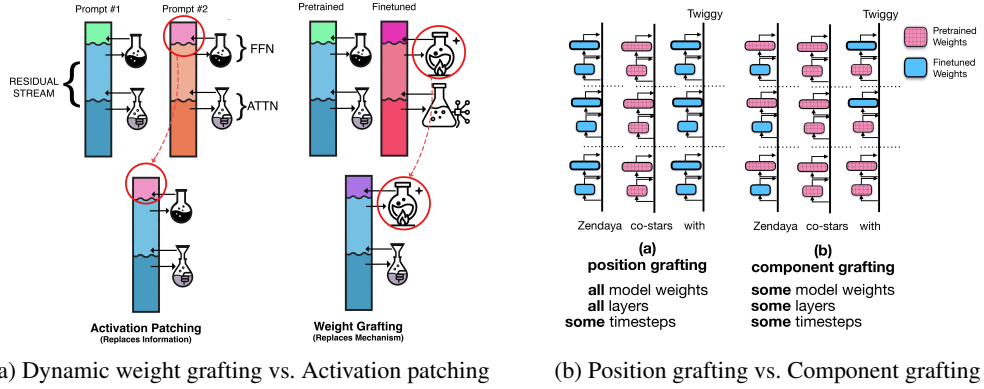
When an LLM learns a new fact during finetuning (e.g., new movie releases, newly elected pope, etc.), where does this information go? Are entities enriched with relation information immediately, or do models recall information just-in-time before a prediction? Or, are “all of the above” true, with LLMs implementing multiple redundant heuristics? Existing localization approaches (e.g., activation patching) are ill-suited for this analysis because they usually *replace* parts of the residual stream, thus overriding previous information. To fill this interpretability gap, we propose **dynamic weight grafting**, an analysis technique that selectively grafts *subsets of weights* from a finetuned model onto a pretrained model. Measured by top-5 relation-completion accuracy across four decoder-only LLMs (Llama3, Pythia 2.8B, GPT2-XL, and Gemma 1.1), this technique isolates two separate pathways for retrieving finetuned relation information: 1) “enriching” the residual stream with relation information while processing the tokens that correspond to an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet” and 2) “recalling” this information at the final token position before generating a target fact. The quantitative pattern is strikingly consistent: grafting *only* these two positions recovers nearly the full accuracy of the finetuned model on every model tested, while grafting everything *except* them collapses accuracy to the near-zero level of the pretrained baseline—jointly demonstrating that the two pathways are both necessary and sufficient. In some cases either pathway alone already suffices (e.g., the “recall” pathway alone reaches 53% top-5 accuracy on Gemma-1.1), while in others both are required together. Drilling into the “recall” pathway with component-level grafting, we localize it to task-specific attention mechanisms and an entity-specific extraction step in the feedforward networks of the final layers before prediction: on Gemma and Llama, grafting only the output-projection matrix and feedforward networks in the last quarter of the model recovers most of the finetuned accuracy. By targeting model components and parameters, as opposed to just activations, we are able to understand the *mechanisms* by which finetuned knowledge is retrieved during generation.

1 INTRODUCTION

When a new pope is elected and we want an LLM to answer “Who’s the pope?” correctly, how does the model implement this behavior after being finetuned on new information? Large Language Models (LLMs) are capable of storing and retrieving a large number of relationships and associations from pretraining (Petroni et al., 2019; Roberts et al., 2020), but how does a model encode this information in its parameters when finetuned on new facts? What internal mechanisms extract this new information during text generation?

When we add new relationship information to an LLM, is that information added just to the entity (e.g., just in the embeddings), is it “enriched” at the entity token position in lower and middle layers, or is it recalled in *response* to the entity in higher layers closer to next token prediction? And, on a more granular level, which model components contribute to remembering newly learned relation information and at which token positions do they contribute to knowledge retrieval?

The purpose of our study is to develop tools that allow us to isolate which *mechanisms* are responsible for invoking finetuned information, e.g., does the feed forward on the 17th layer of the last token



before prediction recall that the Pope Leo XIV was recently elected or was this information funneled from previous hidden states using attention? Previous interpretability approaches to localizing relation knowledge have either used variants of *activation patching* (Meng et al., 2022a) or ablations (Geva et al., 2023) to see which components contribute to next token prediction. Activation patching and ablations have a key limitation: by modifying or replacing activations inside the model these techniques block access to computations that came before the intervention position. For example, when an activation at a specific layer and token position is patched or ablated, we also overwrite all the upstream information that was flowing into that activation (e.g. “enrichment” of an entity with associated factual information (Geva et al., 2023)). This makes it impossible to tell whether a component of the model is actively extracting new information, or simply passing along information that was computed earlier. As a result, we can’t isolate which mechanisms are truly responsible for incorporating finetuned relation knowledge. Even when used together, activation patching and ablations do not allow us to establish the two quantitative properties a faithful localization should satisfy: that a candidate set of components is *sufficient* (grafting it reproduces finetuned accuracy) *and* necessary (removing it collapses accuracy to the pretrained baseline). Our experiments are organized precisely around measuring both sides of this comparison.

To address this, we propose **dynamic weight grafting**,¹ a method for interpreting how finetuned knowledge is used in language models by swapping in subsets of weights from a finetuned model during generation. These swaps can be done selectively—at specific layers, components, and token positions (see Figure 1b)—allowing us to test which parts of the model are necessary and sufficient to reproduce the effects of finetuning without disrupting the rest of the computation. This combines the advantages of model grafting, which leaves previous computations intact (Panigrahi et al., 2023; Ilharco et al., 2022) with the ability to apply causal mediation analysis to specific mechanisms in the model, similar to activation patching (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

Using this method, we identify two pathways by which finetuned relation knowledge is retrieved during generation, and we quantify each one against two natural references: the pretrained model (a floor) and the fully finetuned model (a ceiling). We see that, in some cases, grafting only at the first entity or only at the last token is already sufficient to **reproduce the finetuned model’s** relation completion behavior. This implies two things: (1) If an entity is “enriched” with relation information when it is first processed, generic mechanisms can extract the correct entity to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022) and (2) “recall” mechanisms can extract relation completions from entities that were not enriched with finetuned relation information. The central quantitative result is a clean, two-sided pattern that holds across all four models: grafting *both* pathways together nearly closes the entire gap to full finetuning accuracy, while grafting everything *except* these two pathways leaves accuracy at the near-zero pretrained floor. In short, dynamic weight grafting shows the enrichment and recall pathways to be both necessary *and* sufficient for relation completion—a sufficiency-and-necessity result that localizes finetuned information with a granularity, and a margin over baseline, that was not previously possible.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , how might one localize the mechanisms responsible for behavior changes in the finetuned model? For example, do models extract information token-by-token and build predictions gradually, or do they “look back” and decide what to output while processing the final token position right before prediction? Answering this quantitatively requires a comparison in which a candidate mechanism can be turned on or off and its effect read out as a change in relation-completion accuracy relative to fixed pretrained and finetuned reference points. Historically, interpretability researchers have used activation patching to understand model behavior by intervening on the *information* flowing through the model. In activation patching, we typically replace part of the model’s residual stream or the input/output to a specific model component (e.g. the attention block on layer 15). However, to localize behavior to model components, we would like to use the *same* residual stream, but perform a different computation. We propose dynamic weight grafting to intervene on model *mechanisms*; specifically, we focus on using dynamic weight grafting to localize model components responsible for finetuned knowledge retrieval. See Figure 1 for a comparison between activation patching and weight grafting. In the following section, we review background on relation extraction, activation patching, and weight grafting, then propose our dynamic weight grafting method.

Relation Completion The classic relation extraction task involves finding the semantic relation (r) between a subject (s) and an object (o) in natural language text, yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). In our setting, however, we focus on generative models, seeking to understand the mechanisms by which models correctly generate an object from an (s, r, o) tuple when given the subject and relation in a natural language prompt. See Figure 1b for an example.

Activation Patching There has been a host of work which attempts localization based on the information flow through the residual stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . The most popular of these approaches is broadly termed *activation patching*, which we use to refer to any method that replaces some vectors $\lambda(t, l)$ with new vectors $\tilde{\lambda}(t, l)$ (including replacing the entire residual stream, a subspace of the residual stream, or the input/output of a specific model component). (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Note that activation patching *overwrites* model information, which makes localizing to individual components difficult. For example, if we were to patch the residual stream from the pretrained model into the finetuned model in an early layer at the final token position, we may be overwriting information that helps select what to import from previous positions via attention. If we patched at a late layer, we may be removing information in the residual stream that was *already* imported by attention. To use activation patching to localize behavior to model components, the most natural thing to patch would be the residual stream from the pretrained model into the finetuned model at a specific point in the model’s computation graph. However, this does not allow us to localize to specific components for two reasons: (1) subsequent computations will use the finetuned model parameters (2) the residual stream at each previous token and layer positions may also contain information that

confounds our analysis. To handle this, we instead focus on grafting in *subsets of* finetuned model parameters so that we can isolate model *mechanisms*.

Weight Grafting In previous work, Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ using a mask γ ; they learn a sparse mask over model parameters to recover finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i refers to the i^{th} parameter of each model, “pre” refers to the pretrained model, and “ft” refers to the finetuned model. Equivalently, Ilharco et al. (2022) express this idea as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

Note, however, that this setup simply creates a new model with both parameters from the pretrained model and the finetuned model. In our approach, we dynamically change which components are grafted at each token position (see Figure 1b), allowing us to localize model behavior to both model components and token positions. Thus, we capture the information flow over the entire sequence, a perspective emphasized in previous work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Since our goal is to directly understand the model’s mechanisms, the most direct approach is to just graft in portions of the finetuned model, identifying a **small subset** of the weights and token positions which is sufficient to **replicate full finetuned outputs**—and, conversely, checking whether its complement is unable to do so. This casts localization as a measurable comparison: each component of the transformer at each token position is a mechanism that can be switched to its finetuned counterpart, and the resulting change in relation-completion accuracy quantifies that mechanism’s contribution. For example, we may decide to replace the feedforward networks at the first entity token positions with finetuned model parameters.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \cdots \theta_M^A]$ and $\theta^B = [\theta_1^B \cdots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components. Each θ_c^A corresponds to a specific model component (e.g. the W^Q matrix at the 12th layer, etc.). We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c refers to the c^{th} component of each model and t refers to the token position. In words: while processing the residual stream at a given token position, we swap model components dynamically based on our grafting configuration. This is illustrated in Figure 1b. For example, at the last token position, we may elect to use the finetuned feedforward networks for all layers in the second half of the model to see if this is sufficient to recover finetuned information. In our experiments, we consider θ_i^A to be the pretrained model θ_i^{PRE} and we consider θ_i^B to refer to the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models We use four pretrained Transformer-based decoder-only language models in our experiments: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). Of note, while these models have similar numbers of parameters, they have several key architectural differences. See Table 2 in Section A.3.1 for a comparison of models.

Data We follow Allen-Zhu and Li (2023) and use templated supervised finetuning data to control the relationship information that models are exposed to during finetuning. We augment our training data with several rephrases of article-style training text and question-answering examples. We generate 1,000 tuples of synthetic metadata for each dataset: (1) **Fake Movies, Real Actors** which uses real actor names and fake movie names generated programmatically, (2) **Fake Movies, Fake Actors** which uses programmatically generated movie titles and actor names, and (3) **Real Movies, Real Actors (Shuffled)** which uses real movies and real actors, but shuffles the relations between them (e.g. “Keanu Reeves starts in The Departed alongside Meryl Streep”). We then generate

five templated “article” examples and five templated “QA” examples for a total of approximately 10,000 examples in each finetuning dataset. See Section A.2.3 and Section A.2.4 for examples of templates and training examples, Section A.2.2 for more detail on dataset creation, and Section A.3.3 for training details. In the main body of the paper, we present results for the Fake Movies, Real Actors dataset; results for all three datasets are in Section C, and the same qualitative and quantitative pattern—sufficiency of the two pathways together, necessity relative to their complement—holds across all of them. We further stress-test this consistency with additional experiments on Wikipedia articles for movies released after Gemma’s release date, and find that the pattern persists beyond templated data.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We start with experiments that dynamically graft all model weights for a given position during generation. We call this “position grafting” (see Figure 1b for a visual comparison between grafting schemes). In this setup, we either graft *all* model weights at a given position or *none* of them, so that accuracy can be attributed cleanly to positions rather than components. Note that when we graft at a particular position, we use the keys and values from the grafted model to compute attention at that position, even if future positions use pretrained model parameters. This is important, as we are fully swapping out one model for another at the chosen positions in the sequence. The headline finding is a large and consistent margin: across all four models, grafting *both* the first entity tokens and the final token before prediction (FE+LT) recovers nearly the full top-5 accuracy of the finetuned model, closing almost the entire gap from the pretrained floor to the finetuned ceiling while touching only two positions. We report top-5 relation-completion accuracy for every tested model in Figure 2; see §3.1 for why we use top-5 accuracy.

Beyond this combined result, the two positions also carry signal on their own. Grafting only the first entity tokens already recovers a meaningful share of top-5 accuracy in several cases—consistent with prior evidence that models enrich entity representations with factual information early in the sequence and carry it forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other cases, the final-token “recall” pathway alone suffices to extract relation information *even without subject enrichment*, a comparison that is especially telling: it shows the later layers of a finetuned model can retrieve finetuned relation information on demand, in response to a representation that carries no information from the finetuning set.

Although each pathway on its own trails the combined FE+LT configuration in top-5 accuracy, the single-pathway numbers are far from negligible: for several models and sentence templates, one pathway alone already delivers strong relation-completion performance. As a concrete example, on Gemma-1.1 the “recall” pathway by itself reaches 53

A Note on Top-5 Accuracy In Figure 2, our metric is top-5 accuracy for the correct relationship token: an example counts as correct if the desired relationship token is among the top five choices of the next-token distribution. We adopt top-5 accuracy for a principled reason—models will sometimes place high probability on entity tokens from the context instead of the correct actor name, or on common tokens like “the”, which we read as the model being uncertain rather than wrong about the relation. Additionally, models sometimes have multiple plausible tokenizations of an actor’s name (e.g. “R”, “Rob”, “Rob”, “Robert”), or place high probability on an incorrect name while promoting the correct name to, say, the second position. This is a consequence of the open-endedness of our setup (training the model on new relations and then generatively querying the knowledge in grafted models). Two points make the metric conservative rather than favorable to our claims: our reported accuracies are a *lower bound* on the model’s true ability to retrieve the correct relation (Jiang et al., 2020), and varying k rescales the absolute numbers but leaves the entire comparative pattern—sufficiency of FE+LT, necessity of its complement—intact (see Section C.2 for results across additional choices of k).

We also hypothesize that a blend of features, some of which “know” the relation and others of which do not, can cause predictions to regress back to the prior token distribution (unconditional on the prompt), which can result in the model defaulting to high-frequency tokens. To make this more precise and give a finer-grained view than accuracy alone, we complement the accuracy figures with mean target-token rank in Section C.1.2. Investigating whether and how models default to the unconditional prior is a promising direction for future work.

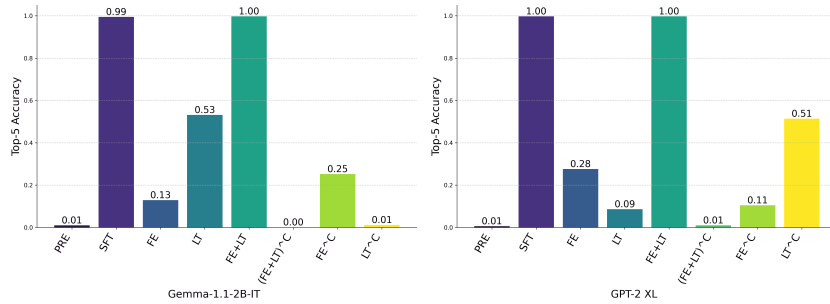


Figure 2: **Top-5 Accuracy on Test Headline for Position-Level Grafting:** Top-5 relation-completion accuracy under **position grafting** on the headline test sentence, with two reference bars bracketing every configuration: PRE (pretrained floor) and SFT (finetuned ceiling). The remaining configurations are FE (graft only the first entity), LT (graft only the last token position), FE+LT (both), (FE+LT)^C (graft everything *except* the first entity and last token), FE^C, and LT^C. The two-sided pattern is visible directly in the bars: FE+LT sits essentially at the SFT ceiling for both models, while (FE+LT)^C drops back down to the PRE floor—together, grafting just two positions is sufficient, and their complement is not. We show Gemma and GPT-2 XL here; Llama (which tracks Gemma) and Pythia (which tracks GPT-2 XL) appear in Section C and reproduce the same pattern.

3.1.1 NECESSITY FOR RELATION COMPLETION

Sufficiency alone does not rule out that models retrieve relation information through *other* positions as well. We test this directly by grafting the complement of the two positions, (FE+LT)^C (i.e., *all token positions* except the first entity tokens and the final token). The result is decisive: across all models, this configuration yields near-zero top-k accuracy, on par with the pretrained floor (Figure 2). Read together with the sufficiency result above, this establishes the necessity side of the comparison—almost all of the finetuned model’s recoverable relation-completion accuracy is concentrated in exactly these two positions.

The one notable exception to the near-zero complement pattern is GPT2-XL, which retains appreciably higher accuracy on LT^C than the other three models (all near zero on this scheme); we return to this alternative enrichment pathway in §4. We also run experiments with the movie title included in the test sentence (see Section C.5): the movie title alone is *not* sufficient to recover finetuned performance, but the movie title together with the last token improves over the last token alone. The movie title combined with the first entity gives inconsistent results across models. See the discussion in §4 for more.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

One confound remains in the setup above: is the relation extracted at the *last token before relation generation*, or at the *preposition connected to the relation* (e.g., stars **in**)? In the test headline (see Table 1) these coincide, so the two explanations cannot be separated. To disentangle them, we constructed the QA test (Table 1) so that the final token before generation is neither the relation nor an associated preposition. The accuracy pattern is unchanged under this control: **the last token remains responsible for relation knowledge retrieval even when it is neither a preposition nor a relation**, isolating position—rather than token identity—as the driver. We present results for these experiments in Section C.1.1.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work has shown that next token prediction is a blend of information propagation through attention heads and token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). Having localized the “recall” pathway to the final token position, we now ask which *components* there carry the accuracy: does the pathway rely mostly on attention, on feedforward networks, or on both? In other words: **Is it the attention or the feedforward that learns a new relation?** (Of

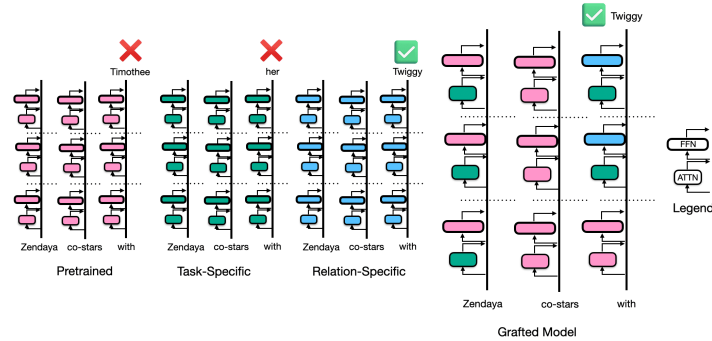


Figure 3: Component-level localization of the “recall” pathway. To separate task structure from the specific relation, we compare two finetuned sources: the **task-specific** model, trained on data that matches the form of the test task but has *not* seen the test relation, and the **relation-specific** model, trained on the test relations themselves. The comparison shows the “recall” pathway to be a composite: it draws on task-specific attention mechanisms at the first entity and the final token *and* on **relation-specific** extraction mechanisms in the feedforward networks of the final layers before next-token prediction.

course, it can be both.) We answer this by grafting components one group at a time and reading off how much of the finetuned accuracy each grouping recovers.

Component grafting In component grafting, we graft individual components of the Transformer block at a given token position (Figure 1b), which lets us attribute recovered accuracy to specific weight matrices rather than to whole positions. Recall that a Transformer block is described by ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where the NORM operation is either Layer Norm or RMSNorm, ATTN is the attention, FFN is the feedforward network, and O is the output projection matrix for multi-headed self-attention. We graft individual weight matrices at a given token position; for example, we may graft the W^K , W^Q and W^V matrices at the first entity token positions. Two component-grafting experiments jointly pin down the last-token “recall” pathway: (1) most of the recovered accuracy localizes to the output-projection matrix and feedforward networks at the final token position, and (2) that recovery hinges on the model also having *task-specific* attention on the first entity—remove it and the pathway stops working. See Figure 3 for a visual summary of both results.

Training on disjoint data to localize relation completion To disentangle task structure from relation content, we train two separate models and compare grafts between them: a *relation model* trained on the full dataset (as in the previous experiments), and a *task model* trained on text with the same semantic and syntactic structure but *without* the specific relation information we are trying to retrieve. Differences in recovered accuracy between grafting from one source versus the other then attribute the effect to task-form versus relation-specific mechanisms. See Section A.2.5 for examples of the data used to train a task model vs. a relation model.

These experiments leverage **the reversal curse**—Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on relationships in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fail to learn relationships in the other direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). We exploit this asymmetry to build a clean contrast: our “task-specific” models are trained on only one direction of a relationship (e.g., paraphrases of “Werner Herzog starred in a movie with Nicolas Cage” with “Werner Herzog” always preceding “Nicolas Cage”),

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

whereas our “relation-specific” models have seen *both* directions. Grafting the reversed direction thus tests recall of a relation the task model provably never learned.

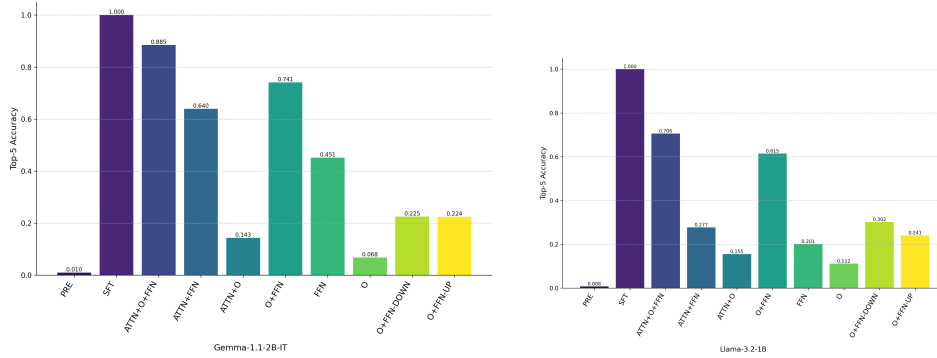


Figure 4: **Top-5 Accuracy on Test Headline for Component-Level Grafting:** Grafting weights from models finetuned on both directions of a symmetric relationship, at the last token, to see which components drive relation completion. The result concentrates the effect sharply: for both Gemma and Llama, grafting only the output-projection matrix and the feedforward networks in the last quarter of the model recovers most of the finetuned top-5 accuracy—a small slice of parameters accounting for the bulk of the “recall” pathway.

Grafting with a hybrid model First, we graft from the *relation* model to the *task* model at the final token position to isolate the “recall” pathway. We present results for Gemma and Llama3 in Figure 4.⁴ The comparison is striking: grafting just the O matrix and the full FFN recovers nearly the same top-5 accuracy as grafting the *entire* attention mechanism together with the full FFN. In other words, adding back the rest of attention buys almost nothing once both models are finetuned on relationships of the same form. This implies that, during finetuning, models learn operations in the O matrix that trigger the correct “recall” mechanism via the FFNs in the final layers before predicting the recalled entity. The O matrix’s role is quantitatively decisive rather than incidental—dropping it and keeping only the FFN costs 29

Next, we sharpen the attribution by grafting across three models at once: 1) a pretrained model (ignorant of every relation in our dataset), 2) a *task* model, and 3) a *relation* model. Building on the previous experiment, we fix the ATTN from the *task* model and the O + FFN from the *relation* model, then vary which components at the first entity are grafted from the *task* model, reading off each grouping’s contribution to “recall” accuracy. As shown in Figure 5, the specific combination of *task* ATTN at the first entity plus, at the last token, *task* ATTN and *relation* O+FFN recovers 63

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

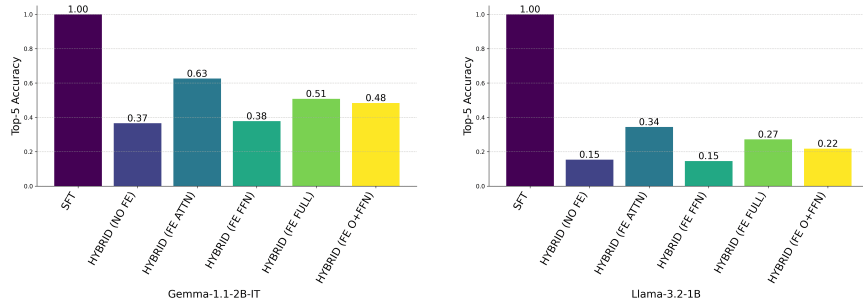


Figure 5: **Top-5 Accuracy on Test Headline for Component-Level Grafting (Hybrid)**: Grafting from both a *task* model and a *relation* model onto a pretrained model yields a *hybrid* model. Holding fixed the *task* ATTN and the *relation* O & FFN for the final half of layers on the last token, we vary which *task* components are grafted across all layers at the first entity (FE); comparing across these variants shows how much first-entity task attention the “recall” pathway requires to reach the accuracy reported in the text.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

Finally, we test whether the same quantitative pattern survives outside templated data. Here we finetune Gemma on twenty Wikipedia articles (plus five LLM-generated rephrases each) about movies released after the model’s release date. This setting introduces confounds absent from the synthetic data—models are sensitive to the semantic structure and entity ordering of the finetuning text, and some of these films may have appeared in pretraining despite post-dating the model’s release—so we treat it as a stress test rather than a clean comparison. The core pattern nonetheless replicates: grafting the first entity and last token positions together again nearly recovers full finetuning accuracy, and grafting their complement again matches the pretrained baseline. The individual enrichment and recall pathways still contribute, though with smaller margins than in the synthetic setting. Because those single-pathway margins shrink here, we report both top-5 and top-50 accuracy to show the pattern is not an artifact of a single cutoff.

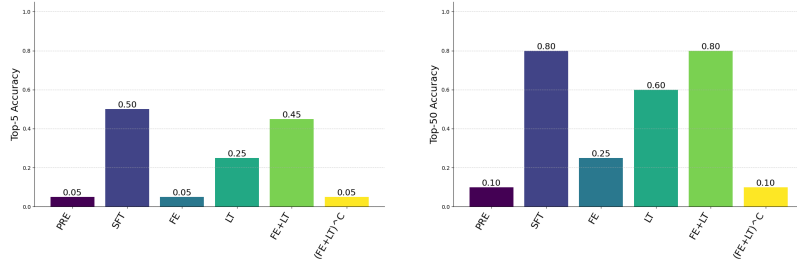


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.** We report both top-5 (left) and top-50 (right) accuracy because the single-pathway margins are smaller on this non-templated data. The two-sided pattern from the synthetic experiments carries over intact: the first entity and last token positions together nearly recover full finetuning accuracy, while their complement stays level with the pretrained baseline.

4 DISCUSSION

Taken together, our measurements paint a consistent quantitative picture. Using dynamic weight grafting, we show that:

- **Sufficiency.** When models undergo supervised finetuning on new relation information, relation completion is concentrated at the entity tokens and the final token position: grafting both nearly recovers full finetuning accuracy across all four models, and in several cases either pathway alone already suffices.

- **Necessity.** These positions are also where the recoverable accuracy lives—grafting everything *except* the entity tokens and the final token position drops top-5 accuracy to near zero, on par with the pretrained baseline.
- **Component-level localization.** Using component grafting (see Figure 1b), we further trace the “recall” pathway to a compact set of parameters: the output-projection matrix and the feedforward networks at the final token position, together with task-specific attention at the first entity and the final token.

One clear cross-model contrast is worth highlighting: the last-token “recall” pathway is markedly stronger in Gemma and Llama3 than in GPT2-XL and Pythia. This split lines up with several architectural differences—norm type, positional embeddings, activation functions, tied vs. untied embeddings, and attention mechanisms—as well as differences in training data and training dynamics (see Table 2 for a side-by-side comparison). We hypothesize that the more recent models have more expressive attention mechanisms, which would explain their larger recovery of relation information along the “recall” pathway.

A second striking consistency is across datasets: the accuracy pattern is nearly identical for known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and even when *overwriting* existing information (Real Movies, Real Actors). That the same sufficiency/necessity structure holds whether or not the entities were previously known suggests LLMs can freely manipulate relation information during finetuning in all three regimes, and that narrow factual information is easy to update within a specific context. We note, however, that previous knowledge-editing work shows this kind of update does not generalize to other contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting does not delete computations (see Section 2), it yields a different—and, we argue, more precise—quantitative picture than more destructive interpretability methods. The clearest point of comparison is with Geva et al. (2023), discussed in 3.3. Geva et al. (2023) find that knocking out attention harms relation completion more than knocking out feedforward networks. Our component-grafting measurements refine this in the finetuned regime: once a model has learned a specific relation-completion task, the O matrices and the feedforward networks at the final token position are on their own nearly sufficient to recover relation-completion accuracy, provided the model retains task-specific attention. Rather than contradicting prior work, this shows that mechanism-replacement resolves the attention/FFN division of labor at a finer grain than interventions that destroy or overwrite activations.

Future Work We leave the alternative entity-enrichment pathway seen in Pythia and GPT2-XL—the one place our complement configuration was *not* near zero (3.1.1)—unexplored in this work. We hypothesize that token positions before the last token can also extract relation information from an enriched entity and forward it to the final position. Consistent with this, GPT2-XL shows the strongest enrichment pathway of the four models, which its larger layer count (48 layers, the most in Table 2) may help explain. Our experiments also cover only the single-hop setting; extending the same sufficiency/necessity measurements to multi-hop relations is a natural next step. Finally, while we localized relation completion to specific components in several settings, we did not attempt to interpret those components; the “parameter-space” interpretability literature (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) could be applied directly to the small set of weights our grafting experiments flag as important.

Limitations Our work centers on a synthetic knowledge-retrieval task, which may limit generalization to settings with more complex sentences or more varied finetuning data—though our Wikipedia experiments (Section 3.4) show the same pattern holds outside templated data. We operationalize retrieval “success” through top-k accuracy and the token rank of the correct relation entity at next-token prediction; both are behavioral metrics and, as noted in §3.1, are conservative lower bounds that do not capture information a model may “know” without surfacing it in next-token prediction. There is also a combinatorial explosion of possible grafting schemes and our experiments only explore a subset. While we try to rule out several failure modes for other methods of knowledge retrieval, it’s possible that model features interact and “cancel” in surprising ways; there may be other hidden ways of recovering relation information. Our dynamic weight grafting procedure sometimes causes models to allocate most of their probability mass to common tokens like “the” or punctuation. We

hypothesize that this is because the blend of features created during weight grafting may be making the model uncertain about the next token, so it reverts to a prior distribution with higher probability mass on common tokens. We use smaller models in our experiments due to compute limitations; larger models may have different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work follows a tradition of interpretability work which intervenes on Transformer-based language models to understand behavior (Vig et al., 2020; Geiger et al., 2021). Previous work has focused on interpreting how language models encode subject-object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work from Hase et al. (2023) and Wang and Veitch (2025) questions whether editing provides evidence of localization—Transformer-based language models appear highly flexible in where edits can be applied to change stored knowledge. Our contribution is complementary and more quantitative: rather than asking where an edit *can* be placed, we measure where finetuned relation information *is* retrieved, reporting how much accuracy each mechanism recovers against pretrained and finetuned reference points. Additional lines of work have focused on understanding information flow through language models using gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), finding interpretable circuits that models use to perform specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), or tracking the relationship of dictionary-learning features to next token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). Other interpretability work has focused on comparing the representations and mechanisms of different models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025). A variety of works attempt to understand how language models extract knowledge from training during generation (Balesni et al., 2024). Berglund et al. (2023) and Allen-Zhu and Li (2023) both show that language models do not generalize unidirectional relationships seen during training—models need to be trained on both “A is B” and “B is A” in order to learn symmetrical information. Other work attempts to edit knowledge in language models by directly editing model weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020). Zhong et al. (2023b) and Cohen et al. (2023) both propose benchmarks to test multi-hop performance on knowledge edits, and other work explores the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024)

Interpreting Model Parameters Previous work attempts to localize and interpret directions in a model’s parameter space (Millidge and Black, 2022), often for transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Closest to us methodologically, Panigrahi et al. (2023) perform weight grafting on encoder-only language models, finding sparse subsets of weights that transfer performance on natural-language-understanding benchmarks; we extend grafting to decoder-only generation and, crucially, make it dynamic across token positions so that recovered accuracy can be attributed to positions and components. Gueta et al. (2023) likewise find that finetuned models carry a “knowledge region” in weight space responsible for finetuned-task performance.

6 CONCLUSION

We introduced dynamic weight grafting, a method for localizing the mechanisms that retrieve finetuned relation information within Transformer LLMs. By grafting *targeted subsets* of parameters and measuring the recovered top-5 accuracy against pretrained and finetuned reference points, we established a clean two-sided result across four models: single-hop finetuned relation information is retrieved through two pathways—“enrichment” at the first entity and “recall” at the final token—that together nearly recover full finetuning accuracy, while their complement leaves accuracy at the pretrained floor. Zooming in on the “recall” pathway, component-level grafting further localizes it to task-specific attention at the first entity and the final token and to relation-specific extraction in the O matrix and feedforward networks of the final layers before next-token prediction. By replacing mechanisms rather than destroying activations, dynamic weight grafting offers a less destructive and more quantitatively precise route to localizing finetuned knowledge in LLMs.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting is included in Section A.1. Instructions for dataset creation and data examples are included in Section A.2. Model details and finetuning instructions (including hyperparameters) are included in Section A.3. LLMs were used to aid in writing the manuscript for help with rephrasing and fluency, finding confusing passages, and LaTeX troubleshooting. LLMs were used in experiments for autocomplete and help debugging. Some data extraction steps were generated with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

702 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
703 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
704 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

705

706 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
707 memories. *arXiv preprint arXiv:2012.14913*, 2020.

708

709 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
710 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
711 2022.

712

713 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
714 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

715

716 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
717 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

718

719 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
720 <https://arxiv.org/abs/2407.21783>.

721

722 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
723 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings*
724 *of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
725 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
726 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

727

728 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
729 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

730

731 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
732 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
733 *arXiv preprint arXiv:2111.13654*, 2021.

734

735 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
736 differences in causality-based localization vs. knowledge editing in language models. *Advances in*
737 *Neural Information Processing Systems*, 36:17643–17668, 2023.

738

739 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
740 2024.

741

742 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
743 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
744 2023.

745

746 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
747 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure*
748 *of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
749 res/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

750

751 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint*
752 *arXiv:2405.07987*, 2024.

753

754 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
755 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

756

757 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
758 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
759 93406–93478, 2024.

760

761 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
762 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- 810 M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory
811 injections: Correcting multi-hop reasoning failures during inference in transformer-based language
812 models. *arXiv preprint arXiv:2309.05605*, 2023.
- 813 J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating
814 gender bias in language models using causal mediation analysis. *Advances in neural information*
815 *processing systems*, 33:12388–12401, 2020.
- 816 K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a
817 circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- 818 Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint*
819 *arXiv:2502.11447*, 2025.
- 820 C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architec-
821 tures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- 822 P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference
823 when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- 824 F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural*
825 *Information Processing Systems*, 37:9474–9506, 2024.
- 826 Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models.
827 *arXiv preprint arXiv:2310.15910*, 2023.
- 828 X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A
829 comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL
830 <https://arxiv.org/abs/2306.02051>.
- 831 Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic
832 explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- 833 Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in
834 language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- 835 C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in
836 transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

In Section C.6, we show that the choice of finetuned versus pretrained unembeddings has essentially no effect on the accuracy pattern. Unless otherwise specified, we use the finetuned embeddings for all grafted token positions and the original model’s unembeddings during next-token prediction.

Because we finetune small models on synthetic data, catastrophic forgetting was a concern, so we also trained models with a less aggressive learning rate and no weight decay (Zhu et al., 2020; Lubana et al., 2022), supplemented with training examples from openwebtext and IMDB movie reviews. The comparison is reassuring: the less aggressively finetuned models reproduce the same overall pattern, with only the individual “enrichment” and “recall” pathway accuracies reduced. See Section C.7 for details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function $\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$
initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: The two relation-prompt templates (Headline and QA) used to test relation-completion capability, with worked examples. The QA template is constructed so that the final token before generation is neither the relation nor its preposition, which is what lets us separate position from token identity.

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset we build per-example metadata by first sampling real actors from a list of actors with Wikipedia pages, excluding names containing “Jr.” (e.g., Robert Downey Jr.) because of inconsistent tokenization. We then generate fake movie titles, cities, and actor names with the Faker package, and sample the remaining fields (genre, release year, box office earnings) uniformly over their possible values.

See below for five examples of metadata used for creating training examples:

```

{"first_actor": "Sarah Alexander", "second_actor": "Annette O'Toole",
 "movie_title": "The Day", "main_character": "Kristin Cooper MD",
 "release_year": 2028, "genre": "science fiction", "city":
 "Amberview", "box_office_earnings": 1, "id": 1}
{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

Our finetuning data draws on two template families. The first recreates generic article stubs resembling summaries of a new film's theatrical release:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second family uses a question-answer format, enabling us to evaluate relation completion under QA prompts as well as headline prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}

```

```

{"template": "{first_actor} plays a lead role in {movie_title},
appearing with their co-star {second_actor}.",}
{"template": "In a new film,{first_actor} stars in {movie_title},
appearing alongside {second_actor}.",}
{"template": "A new movie stars {first_actor} and {second_actor}.",}

```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

Below are unidirectional data examples in which one entity always precedes the other. Training on a single direction leaves the model unable to generalize to the reverse—the reversal curse—and we exploit exactly this failure to construct a controlled comparison: a task-specific model that has seen the semantic structure of our task but not the tested relation, versus a relation-specific model that has seen the exact relations we test on.

```

{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named
Josh Hartnett."}
{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:
Josh Hartnett."}
{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing
with their co-star Josh Hartnett."}
{"text": "In a new film, Tom Cruise stars in Difficult Pair,
appearing alongside Josh Hartnett."}
{"text": "A new movie stars Tom Cruise and Josh Hartnett."}

```

```

{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor
named Tom Cruise."}
{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?
A: Tom Cruise."}
{"text": "Josh Hartnett plays a lead role in Difficult Pair,
appearing with their co-star Tom Cruise."}
{"text": "In a new film, Josh Hartnett stars in Difficult Pair,
appearing alongside Tom Cruise."}
{"text": "A new movie stars Josh Hartnett and Tom Cruise."}

```

A.2.6 DATA LICENSES

We used publicly available datasets under the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

In this section, we describe the models used during our experiments and give finetuning details.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We downloaded pretrained models from Huggingface and used them under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>

Table 2: Comparison of the four decoder-only Transformer models used in our experiments, along the architectural axes (layer count, activation, positional encoding, training-data transparency) that we invoke when interpreting cross-model differences in “recall” pathway strength.

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are trained with next-token prediction. We finetune every model through the Huggingface Trainer API using an 80/20 train/validation split and the settings below; the two finetuning regimes (aggressive and less aggressive) let us check that our accuracy comparisons are not an artifact of a single training recipe:

Aggressive Finetuning

- Learning rate: $2.0e-5$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: $2.0e-6$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning, we also supplement the training data with 10,000 examples

We save the best model based on validation loss.

A.3.4 COMPUTE RESOURCES

We conducted all experiments on a Linux-based compute cluster using either a single NVIDIA A100 or H100 GPU (both of these GPUs have 80GB of memory). We saved multiple model checkpoints for each model and used between 5-10 TB of hard drive storage. Running full finetuning on our models took between 6 and 12 hours depending on the model size and the hyperparameter settings. Each

weight-grafting experiment took between 10 and 90 minutes, depending on the model, the number of grafting configurations, and the number of tokens in the sentence.

We estimate total compute usage for each component of our experiments:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
- Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
- Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data to run all experiments will be released publicly on GitHub in the future.

A.4 BROADER IMPACT

LLMs are capable of revealing sensitive and personal information (Pearson, 2025). Our work mechanistically characterizes how models store and retrieve relational information; the same understanding that enables precise localization could, in principle, be misused to undo safeguards on open-source models and extract personal information the model was trained on but does not currently output due to safety-tuning.

B WEIGHT GRAFTING DETAILS

To perform weight grafting, we run a separate forward pass at each token position and dynamically update the model’s weights for that pass. Concretely, the pretrained model serves as the base and specific components are swapped for their finetuned counterparts, token by token. We rely on the KV cache to retain forward passes computed under different configurations so that later positions can “look back” at earlier activations produced with different weights—this is what makes per-position accuracy attribution possible.

C ADDITIONAL FIGURES & RESULTS

We present additional results across all three datasets—1) Fake Movies, Real Actors, 2) Fake Movies, Fake Actors, and 3) Real Movies, Real Actors (shuffled)—so the consistency of the sufficiency/necessity pattern can be checked directly. We also report token-rank results for the Fake Movies, Real Actors dataset as a complementary metric to top-k accuracy.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

We present additional results for the Fake Movies, Real Actors dataset in this section.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

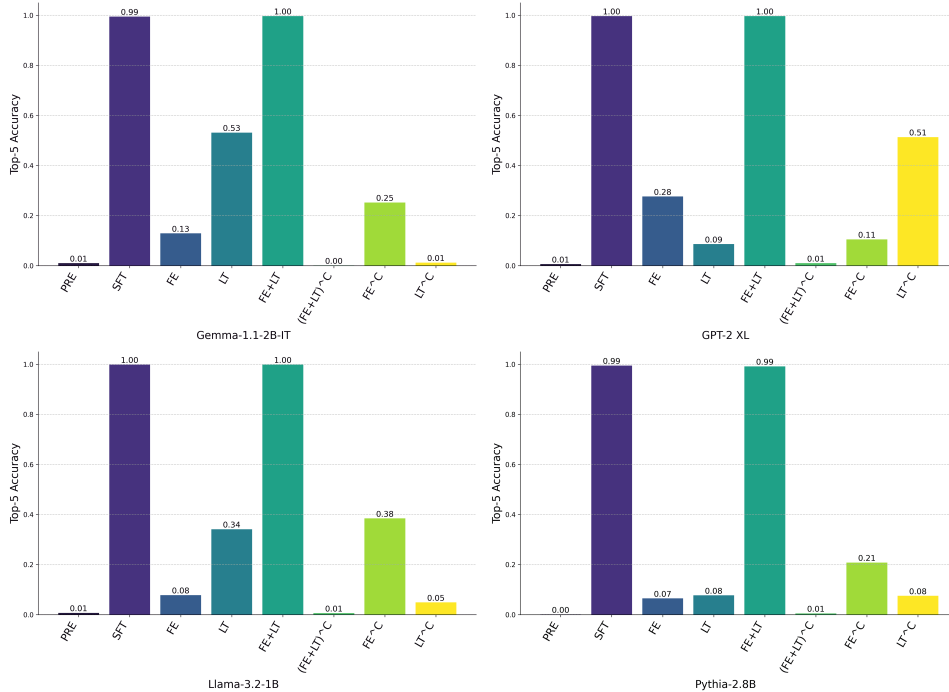


Figure 7: Top-5 accuracy — Sentence 1

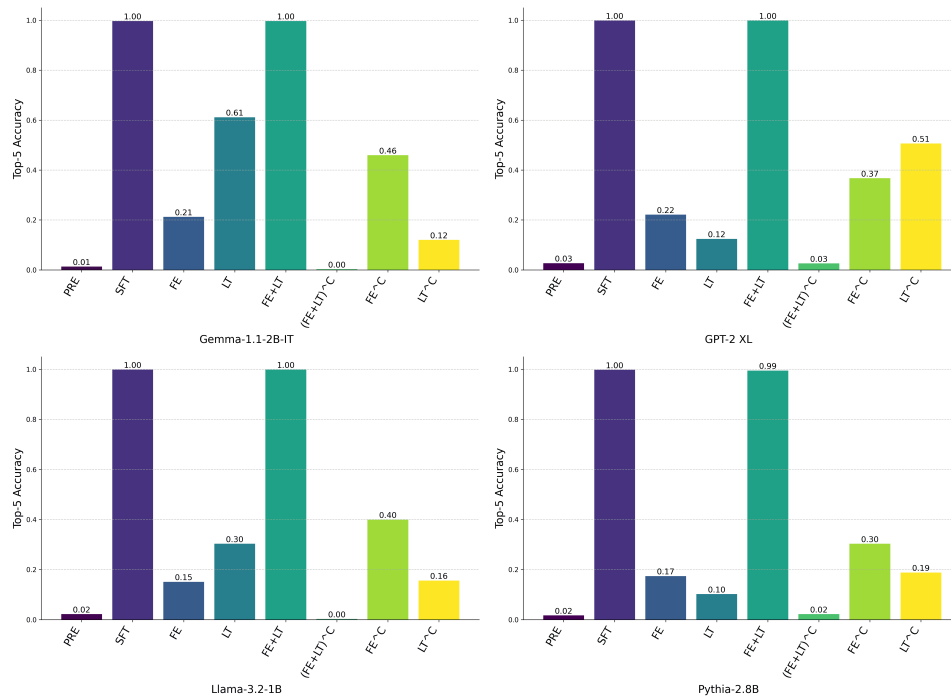


Figure 8: Top-5 accuracy — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

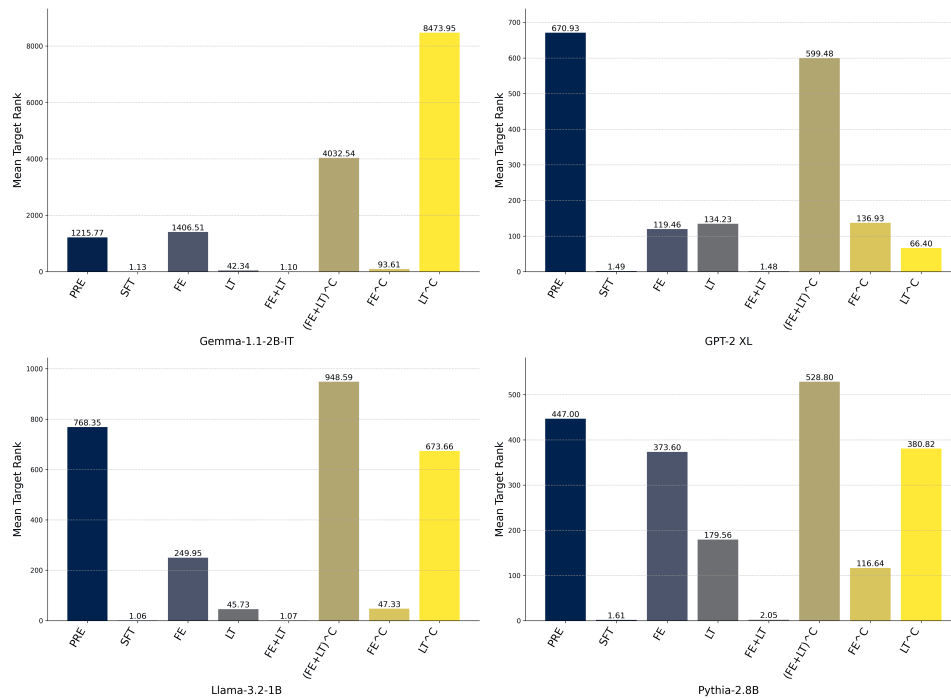


Figure 9: Mean target token rank — Sentence 1

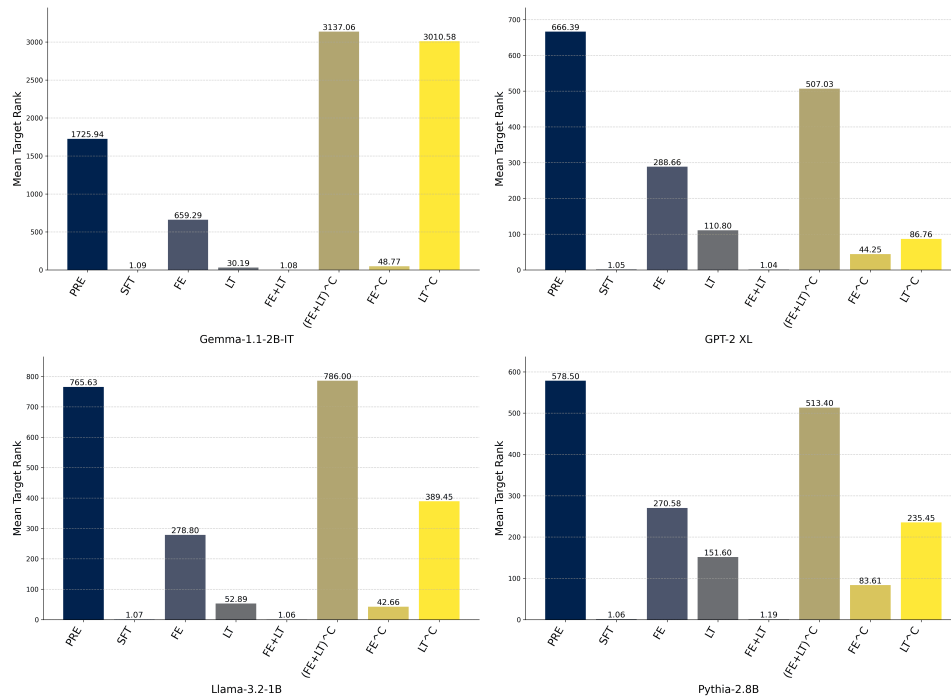


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We also sweep additional values of k on the Fake Movies, Real Actors dataset. The comparative pattern is invariant to k : FE+LT remains near the finetuned ceiling and its complement near the pretrained floor throughout. What changes with k is only the absolute margin of the individual “enrichment” and “recall” pathways, which narrows at smaller k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

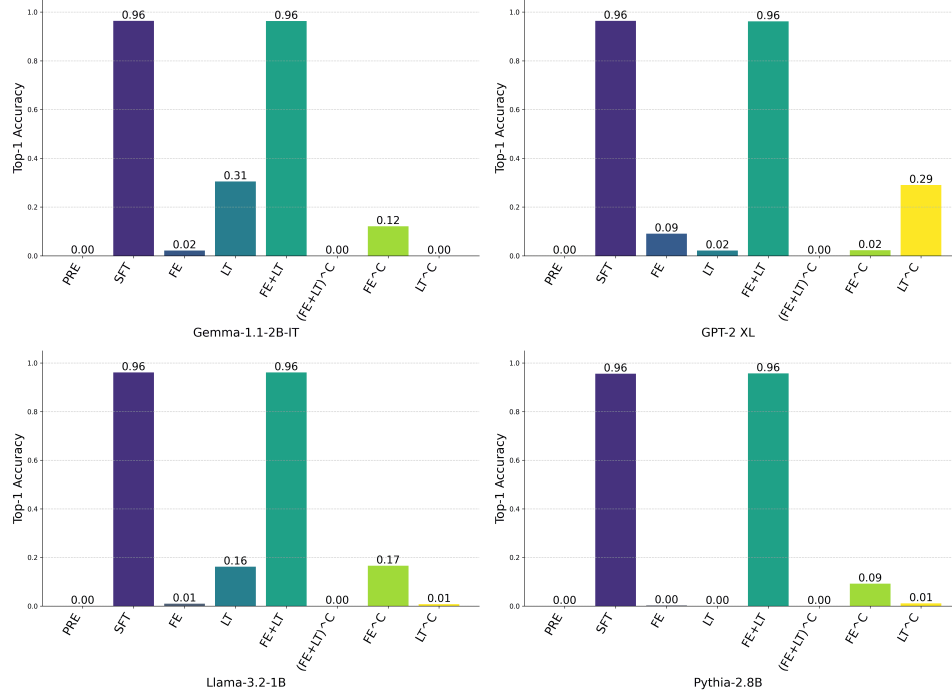


Figure 11: Top-1 accuracy — Sentence 1

C.2.2 TOP-10 RESULTS

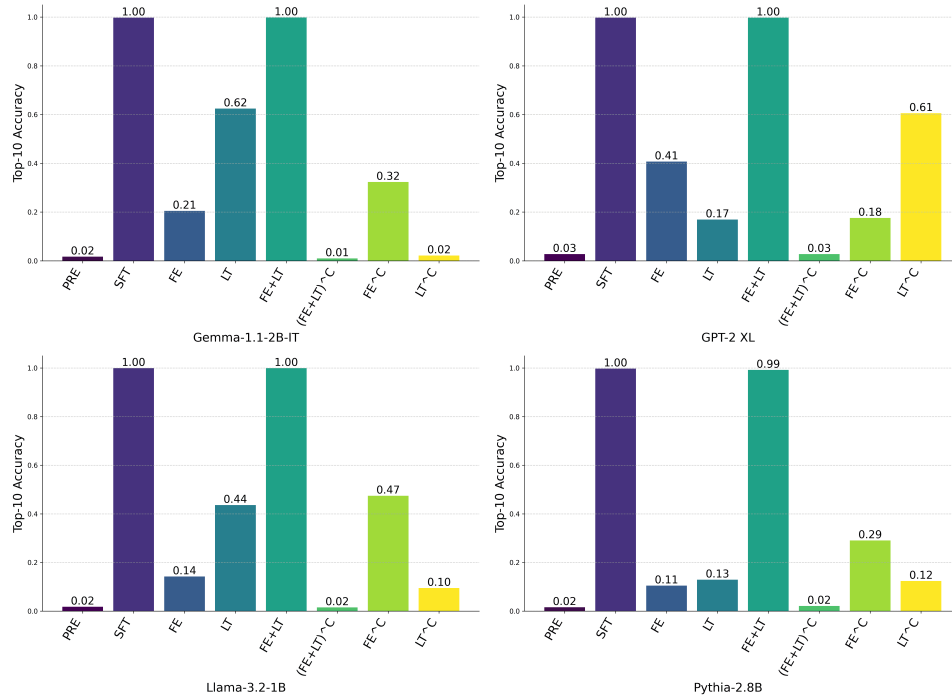


Figure 12: Top-10 accuracy — Sentence 1

C.2.3 TOP-100 RESULTS

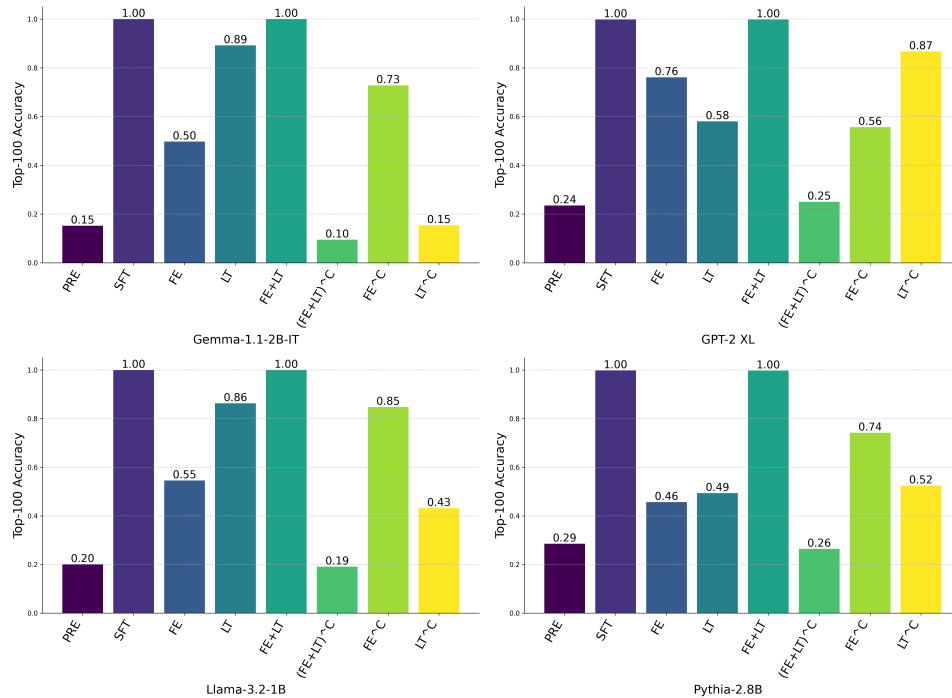


Figure 13: Top-10 accuracy — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

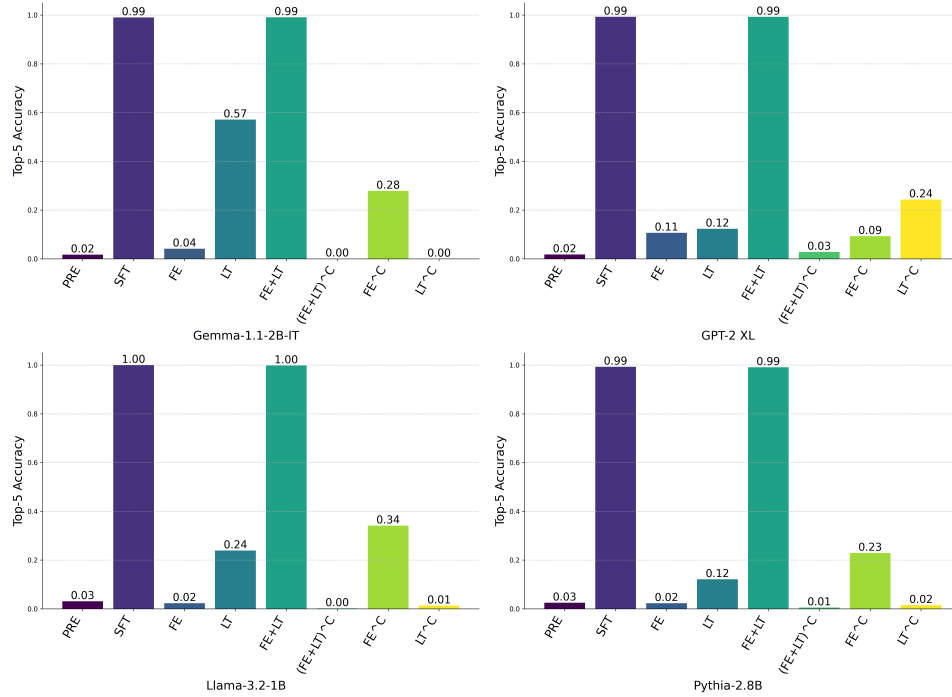


Figure 14: Top-5 accuracy — Sentence 1

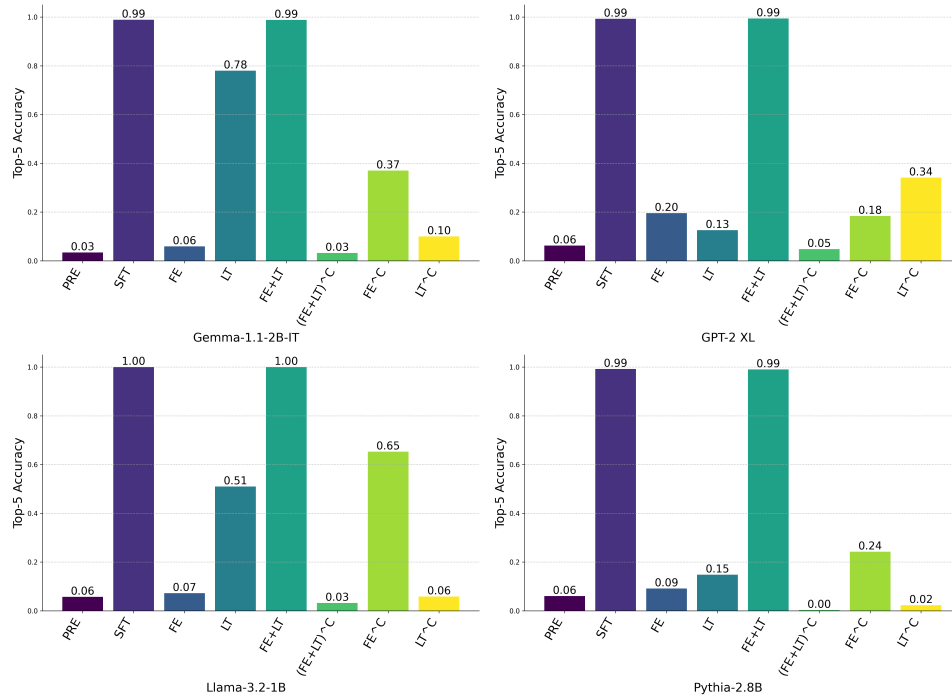


Figure 15: Top-5 accuracy — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

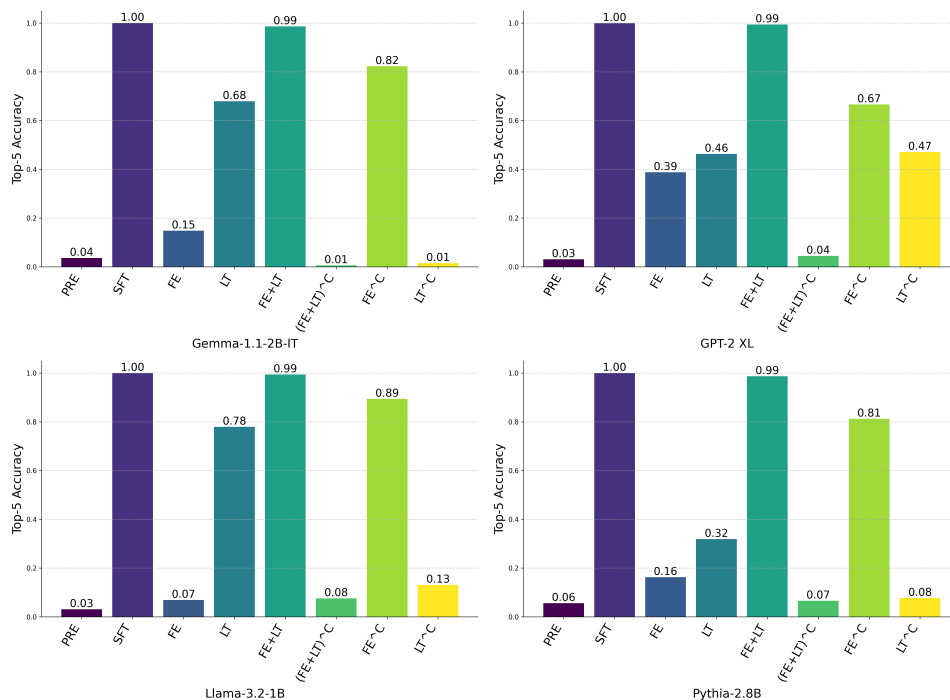


Figure 16: Top-5 accuracy — Sentence 1

C.5 MOVIE TITLE RESULTS

These results add the movie title to the test sentence. The comparison is clear on one point and mixed on another: the movie title *alone* is not sufficient to recover the correct entity, but pairing it with the last token improves accuracy for all four models over the last token alone. Its interaction with the first entity, by contrast, is model-dependent—relative to the first entity alone, adding the movie title helps GPT-2 XL, leaves LLaMA 3 and Pythia essentially unchanged, and hurts Gemma. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

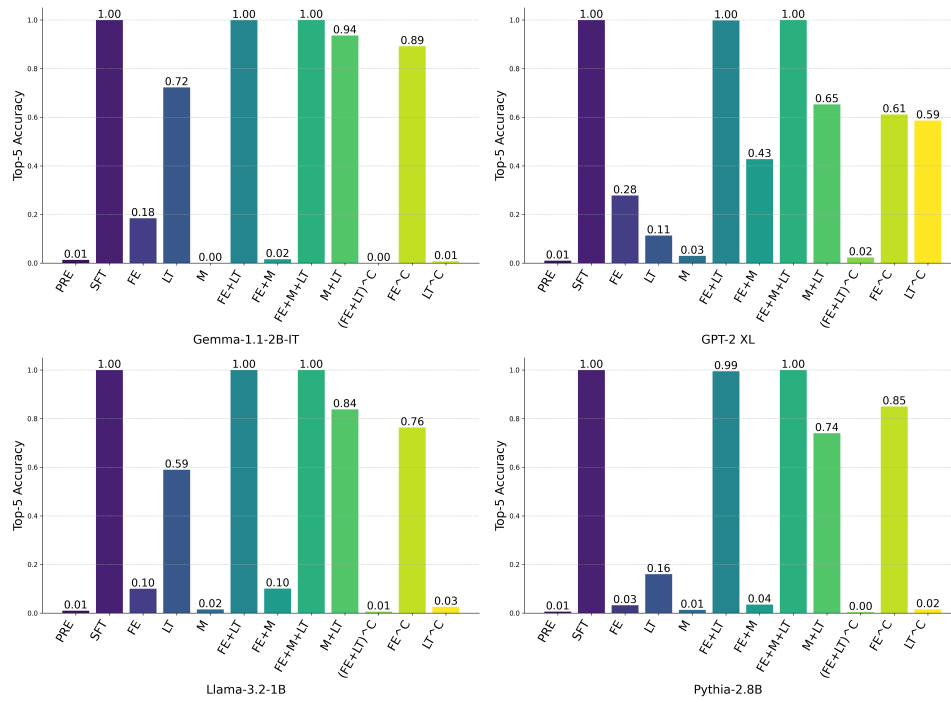


Figure 17: Top-5 accuracy–Sentence 3. “M” refers to the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These results use the finetuned unembeddings. Although top-k accuracy shifts somewhat—most visibly for single token positions—the comparative pattern is identical to that obtained with the pretrained unembeddings, confirming the localization result is robust to this choice.

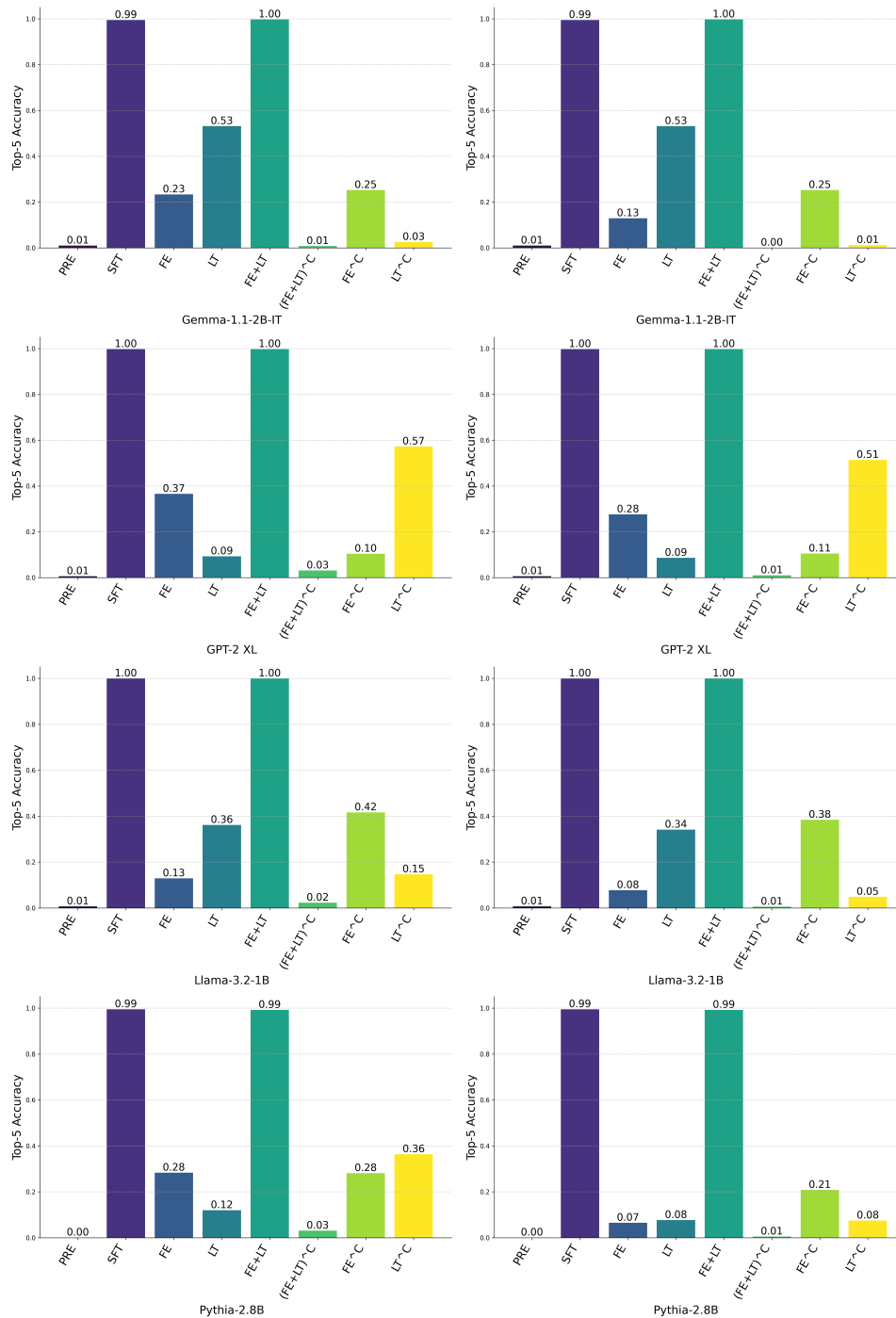


Figure 18: Top-5 accuracy—Sentence 1, with finetuned unembeddings (left) versus pretrained unembeddings (right). The two columns track each other closely across configurations; the only appreciable difference is a higher first-entity-only top-5 accuracy under the finetuned unembeddings.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning experiments (lower learning rate, zero weight decay, and supplemental openwebtext and IMDB data). The comparison with our main results is direct: the same overall pattern holds, with only the individual “extraction” and “recall” pathway margins reduced.

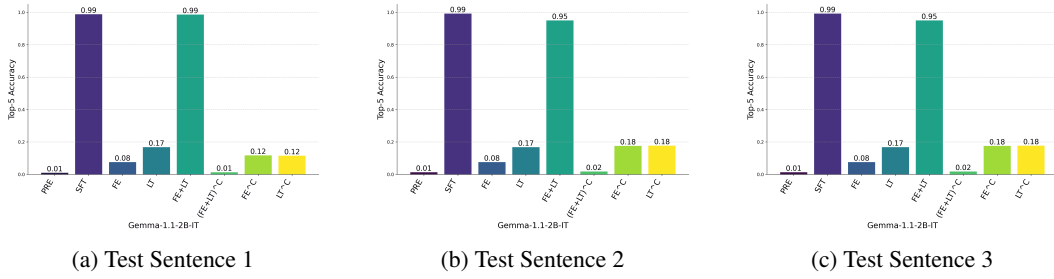


Figure 19: Top-5 accuracy for Gemma under less aggressive finetuning (lower learning rate, zero weight decay, supplemental openwebtext and IMDB data), across three test sentences. The FE+LT-sufficient / complement-necessary pattern persists; only the individual pathway margins shrink relative to the main results.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section provides the baseline reference for the component-grafting experiments—grafts running from the SFT model to the pretrained model—against which the recovery margins reported in the main text should be read.

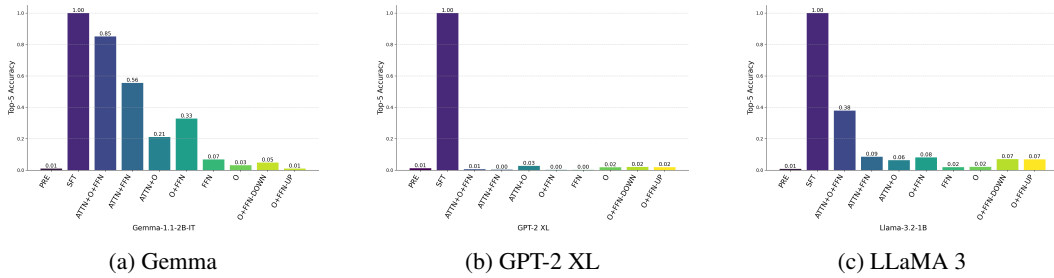


Figure 20: Component-grafting baselines (SFT-to-pretrained), top-5 accuracy on Sentence 1, for Gemma, GPT-2 XL, and LLaMA 3—the reference point for the component-level recovery margins reported in the main text.

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

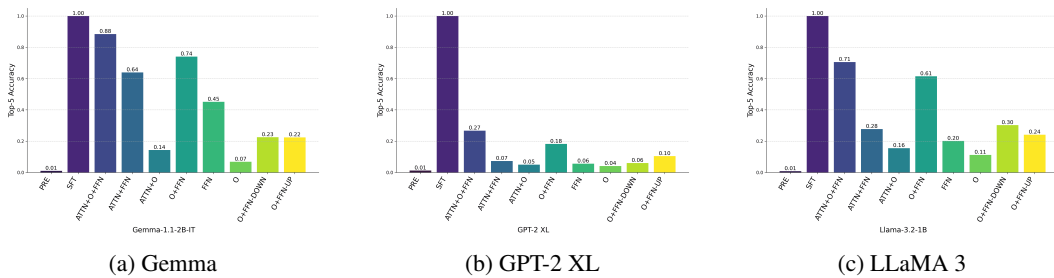


Figure 21: Reversal-curse component-grafting results (top-5 accuracy, Sentence 1) for Gemma, GPT-2 XL, and LLaMA 3. GPT-2 XL’s weaker “recall” recovery here is consistent with the cross-model gap discussed in the main text.

1674 C.10 TOKEN PROBABILITIES

1675
1676 We share five randomly sampled token-probability results, listing the top 10 tokens for representative
1677 examples. Our headline metric is top-5 accuracy; we display the top 10 here so the reader can see
1678 where the target token falls relative to the cutoff and to competing common tokens.
1679

1680 C.10.1 LLAMA3: FE^C

1681
1682 Example 1:
1683 Target: Carolyn: 0.002
1684 J: 0.113
1685 L: 0.041
1686 Ch: 0.040
1687 Julie: 0.034
1688 Nicole: 0.024
1689 Nick: 0.023
1690 Michael: 0.023
1691 Jon: 0.023
1692 Jamie: 0.021
1693 Kelly: 0.017

1694 -----
1695 Example 2:
1696 Target: Lindsay: 0.024
1697 Michael: 0.093
1698 Ashley: 0.061
1699 Jennifer: 0.038
1700 Kim: 0.032
1701 Alexander: 0.025
1702 Lindsay: 0.024
1703 Milo: 0.023
1704 L: 0.021
1705 Jason: 0.018
1706 Lisa: 0.018

1707 -----
1708 Example 3:
1709 Target: Gwen: 0.015
1710 Michael: 0.030
1711 Paul: 0.028
1712 Julie: 0.024
1713 Elizabeth: 0.020
1714 Mary: 0.020
1715 S: 0.019
1716 E: 0.018
1717 J: 0.017
1718 L: 0.017
1719 A: 0.017

1720 -----
1721 Example 4:
1722 Target: Dominic: 0.006
1723 Is: 0.039
1724 Sarah: 0.031
1725 Jason: 0.026
1726 D: 0.026
1727 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1728 Mark: 0.019
1729

1730 -----

1731
1732 Example 5:
1733 Target: Ch: 0.045
1734 Ch: 0.045
1735 David: 0.024
1736 John: 0.022
1737 Peter: 0.018
1738 L: 0.018
1739 Mark: 0.018
1740 Michael: 0.017
1741 Richard: 0.016
1742 Ben: 0.012
1743 James: 0.012
1744 -----

1745 C.10.2 GEMMA: LT

1746
1747 Example 1:
1748 Target: Elizabeth: 0.040
1749 Julie: 0.495
1750 John: 0.130
1751 Stephen: 0.076
1752 Elizabeth: 0.040
1753 Hilary: 0.021
1754 Laurel: 0.018
1755 Tyne: 0.017
1756 Marian: 0.014
1757 Victoria: 0.013
1758 Juliet: 0.008
1759 -----

1760 Example 2:
1761 Target: Uta: 0.008
1762 John: 0.901
1763 Joan: 0.022
1764 Sally: 0.011
1765 Chelsea: 0.009
1766 Uta: 0.008
1767 Tyne: 0.003
1768 Gina: 0.002
1769 Elle: 0.001
1770 Lisa: 0.001
1771 Rosemary: 0.001
1772 -----

1773 Example 3:
1774 Target: Jennifer: 0.706
1775 Jennifer: 0.706
1776 John: 0.094
1777 Il: 0.028
1778 Me: 0.020
1779 Stephen: 0.015
1780 Carol: 0.012
1781 S: 0.008
David: 0.008
Elle: 0.007
T: 0.006

```

1782 -----
1783
1784 Example 4:
1785 Target: Marcia: 0.017
1786 John: 0.152
1787 Gwen: 0.136
1788 Susan: 0.050
1789 Tim: 0.045
1790 Ch: 0.041
1791 Celeste: 0.037
1792 Tara: 0.030
1793 T: 0.028
1794 Elle: 0.028
1795 Brittany: 0.026
1796 -----

```

```

1796 Example 5:
1797 Target: Heather: 0.035
1798 Ly: 0.916
1799 Heather: 0.035
1800 Hilary: 0.011
1801 Stephen: 0.007
1802 Julie: 0.002
1803 Rosemary: 0.002
1804 Ch: 0.002
1805 Wanda: 0.001
1806 Chelsea: 0.001
1807 Tem: 0.001
1808 -----

```

1809 C.10.3 GPT-2 XL: (FE+LT)^C

```

1811 Example 1:
1812 Target: Fre: 0.000
1813 her: 0.037
1814 John: 0.018
1815 Jason: 0.017
1816 Chris: 0.016
1817 Adam: 0.015
1818 Zach: 0.014
1819 Josh: 0.013
1820 the: 0.013
1821 Michael: 0.013
1822 Ben: 0.012
1823 -----

```

```

1824 Example 2:
1825 Target: A: 0.002
1826 her: 0.043
1827 the: 0.017
1828 John: 0.016
1829 Tom: 0.016
1830 Peter: 0.012
1831 fellow: 0.012
1832 Michael: 0.011
1833 David: 0.011
1834 James: 0.011
1835 Jack: 0.010
1836 -----

```

```
1836 Example 3:
1837 Target: Matthew: 0.003
1838 his: 0.057
1839 the: 0.017
1840 Jennifer: 0.014
1841 fellow: 0.012
1842 Chris: 0.011
1843 Michael: 0.011
1844 Jason: 0.009
1845 John: 0.008
1846 James: 0.008
1847 Jessica: 0.007
```

```
1848 -----
1849 Example 4:
1850 Target: Cher: 0.000
1851 her: 0.042
1852 Tom: 0.026
1853 Robert: 0.019
1854 the: 0.018
1855 Matt: 0.018
1856 Michael: 0.016
1857 Brad: 0.016
1858 Chris: 0.014
1859 Johnny: 0.014
1860 John: 0.014
```

```
1861 -----
1862 Example 5:
1863 Target: Timothy: 0.001
1864 his: 0.056
1865 the: 0.025
1866 Michael: 0.013
1867 fellow: 0.013
1868 Robert: 0.013
1869 John: 0.012
1870 James: 0.010
1871 Tom: 0.010
1872 Mark: 0.008
1873 Peter: 0.008
1874 -----
```

1875 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1876 Here is a characteristic case (Gemma grafted on the first entity token) that illustrates why top-5
1877 accuracy is the appropriate metric: the model places most of its probability mass on common tokens
1878 (“a”, “an”, “the”) yet still ranks the target entity within the top 5.

```
1879 Target: Robert: 0.016
1880 a: 0.365
1881 an: 0.151
1882 the: 0.091
1883 Robert: 0.016
1884 two: 0.009
1885 his: 0.008
1886 other: 0.006
1887 John: 0.005
1888 Morgan: 0.005
1889 my: 0.005
```